Twelve Simple Algorithms to Compute Fibonacci Numbers*

Ali Dasdan
KD Consulting
Saratoga, CA, USA
alidasdan@gmail.com

April 13, 2018
(updated on August 30, 2026)

Abstract

The Fibonacci numbers are a sequence of integers in which every number after the first two, 0 and 1, is the sum of the two preceding numbers. These numbers are well known, and the algorithms to compute them are simple enough that they are often used in introductory algorithms courses. In this paper, we present twelve such algorithms together with their time and space complexity analyses. Though very simple, these algorithms illustrate eleven concepts from the algorithms field, ranging from top-down vs. bottom-up dynamic programming to recursion depth, and we say which algorithms illustrate which concept. We also present the results of a small-scale experimental comparison of their runtimes on a personal laptop, where the slowest algorithm takes about four orders of magnitude longer than the fastest. Finally, we provide a list of homework questions for students. We hope that this paper can serve as a useful resource for students learning the basics of algorithms.

*The algorithms presented here, and their implementations in Python, are also available in a public repository [2]. We keep adding new algorithms to that repository as we come across them, so it may well contain more algorithms than the twelve we present in this paper.

1 Introduction

The *Fibonacci numbers* are a sequence F_n of integers in which every number after the first two, 0 and 1, is the sum of the two preceding numbers: $0, 1, 1, 2, 3, 5, 8, 13, 21, \ldots$. More formally, they are defined by the recurrence relation $F_n = F_{n-1} + F_{n-2}$, $n \geq 2$, with the base values $F_0 = 0$ and $F_1 = 1$ [1, 6, 9, 11].

The formal definition of this sequence directly maps to an algorithm to compute the nth Fibonacci number F_n. However, there are many other ways of computing the nth Fibonacci number. This paper presents twelve algorithms in total. Each algorithm takes in n and returns F_n.

Probably due to the simplicity of this sequence, each of the twelve algorithms is also fairly simple. This allows the use of these algorithms for teaching some key concepts from the algorithms field, e.g., see [4] on the use of such algorithms for teaching dynamic programming. This paper is an attempt in this direction. Among other things, the algorithmic concepts illustrated by these algorithms include

- top-down vs. bottom-up dynamic programming [1, 4, 13], illustrated by the top-down `fib1`, `fib2` and `fib9` against the bottom-up `fib3`, `fib10` and `fib11`,
- dynamic programming with vs. without memoization [1, 4, 13], illustrated by `fib1`, which recomputes every subproblem, against `fib2`, `fib9`, `fib10` and `fib11`, which cache them,
- recursion vs. iteration [14, 16, 17], illustrated by three pairs that share a formula: `fib1` and `fib2` against `fib3`, `fib7` against `fib8`, and `fib9` against `fib10`,
- integer vs. floating-point arithmetic [5, 18], illustrated by `fib4`, `fib5` and `fib12` against the nine algorithms that use integer arithmetic only,
- exact vs. approximate results [5, 18], illustrated by those same three algorithms, `fib4` and `fib5` being exact only up to $n = 70$ and `fib12` only up to $n = 78$ on our experimental setting,
- exponential-time vs. polynomial-time [15], illustrated by `fib1` against all the others,

- constant-time vs. non-constant-time arithmetic [12], illustrated by all twelve, in the two columns of Table 1,
- progression from constant to polynomial to exponential time and space complexity [1, 15], illustrated in time by `fib4` and `fib5` (constant), `fib7` through `fib11` (logarithmic), `fib2`, `fib3`, `fib6` and `fib12` (linear), and `fib1` (exponential); in space the grouping differs, most sharply for `fib9`, which uses $O(n)$ where `fib10` and `fib11` use $O(\lg n)$ for the same formula,
- closed-form vs. recursive formulas [19], illustrated by `fib4` and `fib5`, and by the matrix-based `fib6`, `fib7` and `fib8`, against the recurrence-based `fib1`, `fib2`, `fib3`, `fib9`, `fib10`, `fib11` and `fib12`,
- repeated squaring vs. linear iteration for exponentiation [1], illustrated by `fib6` against `fib7` and `fib8`, which raise the same matrix to the same power, and
- recursion depth [14, 16, 17], illustrated by `fib1` and `fib2`, whose depth grows linearly with n, against `fib7` and `fib9`, whose depth grows logarithmically.

Given the richness of the field, we expect that computing these numbers will go on yielding further concepts to illustrate.

We present each algorithm as implemented in the Python programming language (so that they are ready-to-run on a computer) together with their time and space complexity analyses. We also present a small-scale experimental comparison of these algorithms. We hope students with an interest in learning algorithms may find this paper useful for these reasons. The simplicity of the algorithms should also help these students to focus on learning the algorithmic concepts illustrated rather than struggling with understanding the details of the algorithms themselves.

Since the Fibonacci sequence has been well studied in math, there are many known relations [11], including the basic recurrence relation introduced above. Some of the algorithms in this study directly implement a known recurrence relation on this sequence. Some others are derived by converting recursion to iteration. In [7], even more algorithms together with their detailed complexity analyses are presented.

Interestingly, there are also closed-form formulas on the Fibonacci numbers. The algorithms derived from these formulas are also part of this study.

However, these algorithms produce approximate results beyond a certain n due to their reliance on the floating-point arithmetic.

For convenience, we call an algorithm *exact* if it returns exactly F_n, and *approximate* otherwise.

2 Preliminaries

These are the helper algorithms or functions used by some of the twelve algorithms. Each of these algorithms are already well known in the technical literature [1]. They are also simple to derive.

Note that $m = [[a, b], [c, d]]$ in the Python notation means a 2x2 matrix

$$m = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \tag{1}$$

in math notation. Also, each element is marked with its row and column id as in, e.g., $m[0][1]$ in Python notation means $m_{01} = b$ in math notation.

Since we will compute F_n for large n, the number of bits in F_n will be needed. As we will later see,

$$F_n = \left\lfloor \frac{\varphi^n}{\sqrt{5}} \right\rceil \text{ and } \varphi = \frac{1 + \sqrt{5}}{2} \approx 1.618033 \tag{2}$$

where φ is called the *golden ratio* and $\lfloor \cdot \rceil$ rounds its argument. Hence, the number of bits in F_n is equal to $\lg F_n \approx n \lg \varphi \approx 0.7n = \Theta(n)$.

In the sequel, we will use $A(b)$ and $M(b)$ to represent the time complexity of adding (or subtracting) and multiplying (or dividing) two b-bit numbers, respectively. For fixed precision arguments with constant width, we will assume that these operations take constant time, i.e., $A(b) = M(b) = O(1)$; we will refer to this case as *constant-time arithmetic*. For arbitrary precision arguments, $A(b) = O(b)$ and $M(b) = O(b^2)$, although improved bounds for each exist [12]; we will refer to this case as *non-constant-time arithmetic*. The non-constant case also applies when we want to express the time complexity in terms of bit operations, even with fixed point arguments. In this paper, we will vary b from 32 to $F_{10,000}$, which is around 7,000 bits per Eq. 2.

What follows next are these helper algorithms, together with their description, and their time complexity analyses in bit operations.

- `is_odd(n)` and `is_even(n)` in Alg. 1: Two functions to test the parity of an integer n. `is_odd` masks off all but the lowest bit of n, which is 1 exactly when n is odd, and `is_even` negates that. Both take constant time even with non-constant-time arithmetic, since only the lowest bit is ever examined. They are used by `negafib` below and by several of the twelve algorithms of Section 3.

- `num_pow_iter(a,n)` in Alg. 2: An algorithm to compute a^n for floating-point a and non-negative integer n iteratively using *repeated squaring*, which uses the fact that $a^n = (a^{(\frac{n}{2})})^2$. This algorithm iterates over the bits of n, so it iterates $\lg n$ times. In each iteration, it can multiply two b-bit numbers at most twice, where b ranges from $\lg a$ to $\lg a^{n-1}$ in the worst case, or where each iteration takes time from $M(\lg a)$ and $M(\lg a^n)$. The worst case happens when the bit string of n is all 1s, i.e., when n is one less than a power of 2. As such, a trivial worst-case time complexity is $O(M(\lg a^n) \lg n)$. However, a more careful analysis shaves off the $\lg n$ factor to lead to $O(M(\lg a^n)) = O(M(n \lg a))$. With constant-time arithmetic, the time complexity is $O(\lg n)$. Wherever an algorithm in Section 3 calls `math.pow(x,y)` from the Python standard library, as `fib4` and `fib5` do, `num_pow_iter(x,y)` can be used in its place. We state that alternative here once rather than repeating it in each algorithm.

- `mat_mul(m1,m2)` in Alg. 3: An algorithm to compute the product $m1 * m2$ of two 2x2 matrices $m1$ and $m2$. This algorithm has eight multiplications, so the total time complexity is $O(M(max(\lg max(m1), \lg max(m2))))$, where $max(m)$ returns the largest element of the matrix m.

- `mat_mul_opt(m1)` in Alg. 3: An algorithm to compute the product $m1 * m2$ of two 2x2 matrices $m1$ and $m2$ when $m2 = [[1, 1], [1, 0]]$, in the Python list notation. This algorithm is an optimized version of `mat_mul(m1,m2)` in Alg. 3. It performs two additions and no multiplications, so the total time complexity is $O(A(\lg max(m1)))$, and not $O(M(\lg max(m1)))$ as for `mat_mul(m1,m2)`. Likewise, wherever an algorithm calls `mat_mul_opt(m)`, as `fib6` does, the general `mat_mul(m,[[1,1],[1,0]])` can be used in its place.

- `mat_pow_recur(m,n)` in Alg. 4: An algorithm to compute m^n of a 2x2 matrix m recursively using repeated squaring. Its time complexity anal-

ysis is similar to that of `num_pow_iter`. As such, the time complexity is $O(M(\lg a))$ where $a = max(r)$. With constant-time arithmetic, the time complexity is $O(\lg n)$.

- `mat_pow_iter(m,n)` in Alg. 5: An algorithm to compute m^n of a 2x2 matrix m iteratively using repeated squaring. Its time complexity is equal to that of its recursive version, i.e., $O(M(\lg a))$ where $a = max(r)$. With constant-time arithmetic, the time complexity is $O(\lg n)$.
- `negafib(m,Fn)` in Alg. 6: An algorithm to return the "negafibonacci" number corresponding to the nth Fibonacci number F_n. A negafibonacci number is a Fibonacci number with a negative index; such numbers are defined as $F_{-n} = (-1)^{n+1}F_n$ [11].
- The function `round(x)` or `[x]` rounds its floating-point argument to the closest integer. It maps to the `math.round(x)` function from the standard math library of Python.

Algorithm 1 `is_odd(`n`)` and `is_even(`n`)`: Two algorithms to test whether the integer n is odd or even.

```
def is_odd(n:int) -> int:
    return n & 1

def is_even(n:int) -> bool:
    return not is_odd(n)
```

Algorithm 2 `num_pow_iter(`a`,`n`)`: An algorithm to compute a^n for floating-point a and non-negative integer n iteratively using repeated squaring.

```
def num_pow_iter(a:int, n:int) -> int:
    n_bits = bin(n)[2:]
    r = 1
    for b in n_bits:
        r = r * r
        if b == '1':
            r *= a
    return r
```

Algorithm 3 `mat_mul`(m_1,m_2): An algorithm to compute the product m_1m_2 of two 2x2 matrices m_1 and m_2. `mat_mul_opt`(m_1) is the optimized version for the case $m_2 = [[1,1],[1,0]]$.

```
def mat_mul(m1:Matrix, m2:Matrix) -> Matrix:
    m = [[0, 0], [0, 0]]
    m[0][0] = m1[0][0] * m2[0][0] + m1[0][1] * m2[1][0]
    m[0][1] = m1[0][0] * m2[0][1] + m1[0][1] * m2[1][1]
    m[1][0] = m1[1][0] * m2[0][0] + m1[1][1] * m2[1][0]
    m[1][1] = m1[1][0] * m2[0][1] + m1[1][1] * m2[1][1]
    return m

def mat_mul_opt(m1:Matrix) -> Matrix:
    m = [[0, 0], [0, 0]]
    m[0][0] = m1[0][0] + m1[0][1]
    m[0][1] = m1[0][0]
    m[1][0] = m1[1][0] + m1[1][1]
    m[1][1] = m1[1][0]
    return m
```

Algorithm 4 `mat_pow_recur`(m,n): An algorithm to compute m^n of a 2x2 matrix m for non-negative integer n recursively using repeated squaring. `mat_pow_opt_recur`(n) is the optimized version for the case $m = [[1,1],[1,0]]$.

```
def mat_pow_recur(m:Matrix, n:int) -> Matrix:
    r = [[1, 0], [0, 1]] # identity
    if n > 1:
        r = mat_pow_recur(m, n >> 1)
        r = mat_mul(r, r)
    if is_odd(n):
        r = mat_mul(r, m)
    return r

def mat_pow_opt_recur(n:int) -> Matrix:
    r = [[1, 0], [0, 1]] # identity
    if n > 1:
        r = mat_pow_opt_recur(n >> 1)
        r = mat_mul(r, r)
    if is_odd(n):
        r = mat_mul_opt(r)
    return r
```

Algorithm 5 `mat_pow_iter`(m, n): An algorithm to compute m^n of a 2x2 matrix m for non-negative integer n iteratively using repeated squaring. `mat_pow_opt_iter`(n) is the optimized version for the case $m = [[1, 1], [1, 0]]$.

```
def mat_pow_iter(m:Matrix, n:int) -> Matrix:
    n_bits = bin(n)[2:]
    r = [[1, 0], [0, 1]] # identity
    for b in n_bits:
        r = mat_mul(r, r)
        if b == '1':
            r = mat_mul(r, m)
    return r

def mat_pow_opt_iter(n:int) -> Matrix:
    n_bits = bin(n)[2:]
    r = [[1, 0], [0, 1]] # identity
    for b in n_bits:
        r = mat_mul(r, r)
        if b == '1':
            r = mat_mul_opt(r)
    return r
```

Algorithm 6 `negafib`(n, F_n): An algorithm to compute the negafibonacci number corresponding to the nth Fibonacci number F_n, using `is_odd` from Alg. 1.

```
def negafib(n:int, Fn:int) -> int:
    if is_odd(n + 1):
        return -Fn
    return Fn
```

3 The Twelve Algorithms

We now present each algorithm, together with a short explanation on how it works, its time and space complexity analyses, and some of the concepts from the algorithms field it illustrates. Each algorithm takes n as the input and returns the nth Fibonacci number F_n. Note that n can also be negative, in which case the returned numbers are called "negafibonacci" numbers, defined as $F_{-n} = (-1)^{n+1}F_n$ [12].

Some of the algorithms use a data structure F to cache pre-computed numbers such that $F[n] = F_n$. For some algorithms F is an array (a mutable list in Python) whereas for some others it is a hash table (or dictionary in Python). In Python, lists and dictionaries are accessed the same way, so the type of the data structure can be found out by noting whether or not the indices accessed are consecutive or not.

Each algorithm is also structured such that the base cases for $n = 0$ to $n = 1$ (or $n = 2$ in some cases) are taken care of before the main part is run. Also note that some of the algorithms in this section rely on one or more algorithms from the preliminaries section.

The listings below are the Python 3 implementations from the repository [2]. There, each algorithm lives in its own module `ad_fibN.py` and its entry point is named simply `fib`; we write `fibN` in the listings so that the twelve can be told apart.

Algorithm 7 `fib1(`n`)`: An algorithm to compute the nth Fibonacci number F_n recursively using dynamic programming. This algorithm uses probably the most well-known recurrence relation defining Fibonacci numbers.

```
def fib_recur(n:int) -> int:
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fib_recur(n - 1) + fib_recur(n - 2)

def fib1(n:int) -> int:
    n0, n = n, abs(n)
    r = fib_recur(n)
    if n0 < 0:
        return negafib(n, r)
    return r
```

3.1 Algorithm fib1: Top-down Dynamic Programming

`fib1` is derived directly from the recursive definition of the Fibonacci sequence: $F_n = F_{n-1} + F_{n-2}$, $n \geq 2$, with the base values $F_0 = 0$ and $F_1 = 1$.

Its time complexity $T(n)$ with constant-time arithmetic can be expressed as $T(n) = T_{n-1} + T_{n-2} + 1$, $n \geq 2$, with the base values $T(0) = 1$ and $T(1) = 2$. The solution of this linear non-homogeneous recurrence relation implies that the time complexity is equal to $T(n) = O(F_n) = O(\varphi^n)$, which is exponential in n.

Its time complexity with non-constant-time arithmetic leads to $T(n) = T_{n-1} + T_{n-2} + A(\lg F_{n-1})$, where the last term signifying the cost of the addition run in $O(n)$ time. The solution of this linear non-homogeneous recurrence relation implies that the time complexity is also equal to $T(n) = O(F_n) = O(\varphi^n)$, which is also exponential in n.

The space complexity, in contrast, is not exponential. The recursion tree has $O(\varphi^n)$ nodes, but only the nodes on the path from the root down to the current node are live at any moment. Hence the recursion depth, and with it the space complexity, is $O(n)$ with constant-time arithmetic and $O(n^2)$ bits with non-constant-time arithmetic [14, 16].

Regarding algorithmic concepts, `fib1` illustrates recursion, top-down dynamic programming, and exponential complexity.

3.2 Algorithm fib2: Top-down Dynamic Programming with Memoization

`fib2` is equivalent to `fib1` except for the addition of memoization. Memoization allows the caching of the already computed Fibonacci numbers so that `fib2` does not have to revisit already visited parts of the call tree.

Memoization reduces the time and space complexities drastically; it leads to at most n additions. With constant-time arithmetic, the time complexity is $O(n)$. The space complexity is also linear.

With non-constant time arithmetic, the additions range in time complexity from $A(\lg F_2)$ to $A(\lg F_{n-1})$. The sum of these additions leads to the time complexity of $O(n^2)$ in bit operations. The space complexity is also quadratic in the number of bits.

Regarding algorithmic concepts, `fib2` illustrates recursion, top-down dynamic programming, and memoization.

Algorithm 8 `fib2(`n`)`: An algorithm to compute the nth Fibonacci number F_n recursively using dynamic programming with memoization (i.e., caching of pre-computed results).

```
def fib_recur(n:int, F:List[int]) -> int:
    if F[n] is None:
        if n == 0:
            F[n] = 0
        elif n == 1:
            F[n] = 1
        else:
            F[n] = fib_recur(n - 1, F) + fib_recur(n - 2, F)
    return F[n]

def fib2(n:int) -> int:
    n0, n = n, abs(n)
    r = fib_recur(n, [None] * (n + 1))
    if n0 < 0:
        return negafib(n, r)
    return r
```

Algorithm 9 `fib3(`n`)`: An algorithm to compute the nth Fibonacci number F_n iteratively in constant storage.

```
def fib3(n:int) -> int:
    n0, n = n, abs(n)
    if n == 0:
        r = 0
    elif n == 1:
        r = 1
    else:
        f2, f1 = 1, 0
        for _ in range(2, n + 1):
            f2, f1 = f2 + f1, f2
        r = f2
    if n0 < 0:
        return negafib(n, r)
    return r
```

3.3 Algorithm fib3: Iteration with Constant Storage

`fib3` uses the fact that each Fibonacci number depends only on the preceding two numbers in the Fibonacci sequence. This fact turns an algorithm designed top down, namely, `fib2`, to one designed bottom up. This fact also reduces the space usage to constant, just a few variables.

The time complexity is exactly the same as that of `fib2`. The space complexity is $O(1)$ with constant-time arithmetic and $O(n)$ with non-constant-time arithmetic.

Regarding the algorithmic concepts, `fib3` illustrates iteration, recursion to iteration conversion, bottom-up dynamic programming, and constant space.

Algorithm 10 `fib4`(n): An algorithm to compute the nth Fibonacci number F_n using the closed-form formula with the golden ratio. It is exact up to $n = 70$ on our experimental setting, and raises beyond $n = 1474$, where φ^n leaves the IEEE 754 double precision range.

```
def fib4(n:int) -> int:
    n0, n = n, abs(n)
    if n == 0:
        r = 0
    elif n == 1:
        r = 1
    else:
        sqrt_5 = math.sqrt(5)
        phi = float(1 + sqrt_5) / 2
        phi_n = math.pow(phi, n)
        psi_n = float(1) / phi_n
        if is_odd(n):
            psi_n = -psi_n
        r = round((phi_n - psi_n) / sqrt_5)
        r = int(r)
    if n0 < 0:
        return negafib(n, r)
    return r
```

3.4 Algorithm fib4: Closed-form Formula with the Golden Ratio

`fib4` uses the fact that the nth Fibonacci number has the following closed-form formula [9, 11]:

$$F_n = \left[\frac{\varphi^n - \psi^n}{\varphi - \psi} \right] = \left[\frac{\varphi^n - \psi^n}{\sqrt{5}} \right] \tag{3}$$

where

$$\varphi = \frac{1 + \sqrt{5}}{2} \approx 1.618033 \tag{4}$$

is the golden ratio,

$$\psi = \frac{1 - \sqrt{5}}{2} = 1 - \psi = -\frac{1}{\varphi} \approx -0.618033 \tag{5}$$

is the negative of its conjugate.

Note that to compute φ^n, this algorithm uses the standard math library function `math.pow(phi,n)`; as noted in Section 2, `num_pow_iter(phi,n)` can be used in its place.

This algorithm performs all its functions in floating-point arithmetic; the math functions in the algorithm map to machine instructions directly. Thus, this algorithm runs in constant time and space (also see [7] for an argument on non-constant time, assuming large n).

For $n > 70$, this algorithm starts returning approximate results. For $n > 1474$, this algorithm starts erroring out as F_n is too large to even fit in a double precision floating point number width. These two limits have different origins. The first depends on the 53-bit significand of the double precision format and on the accuracy of the library's `pow` and `sqrt`, which IEEE 754 does not require to be correctly rounded; it may therefore change with the programming language, the math library, and the computer used, and so may the corresponding limits of `fib5` and `fib12`. The second follows from the exponent range alone: the largest finite double is about 1.8×10^{308}, and since $\lfloor \ln(1.8 \times 10^{308}) / \ln \varphi \rfloor = 1474$, the value φ^n ceases to be representable at $n = 1475$. That limit is the same wherever the IEEE 754 double precision format is used. The general observations still hold.

Regarding the algorithmic concepts, `fib4` illustrates the closed-form formula vs. iteration, integer vs. floating-point computation, and the exact vs. approximate result (or computation).

Algorithm 11 `fib5(`n`)`: An algorithm to compute the nth Fibonacci number F_n using the closed-form formula with the golden ratio and rounding. It is exact up to $n = 70$ on our experimental setting, and raises beyond $n = 1474$, where φ^n leaves the IEEE 754 double precision range.

```
def fib5(n:int) -> int:
    n0, n = n, abs(n)
    if n == 0:
        r = 0
    elif n == 1:
        r = 1
    else:
        sqrt_5 = math.sqrt(5)
        phi = float(1 + sqrt_5) / 2
        phi_n = math.pow(phi, n)
        r = round(phi_n / sqrt_5)
        r = int(r)
    if n0 < 0:
        return negafib(n, r)
    return r
```

3.5 Algorithm fib5: Closed-form Formula with the Golden Ratio and Rounding

`fib5` relies on the same closed-form formula used by `fib4`. However, it also uses the fact that ψ is less than 1, meaning its nth power for large n approaches zero [9, 11]. This means Eq. 3 reduces to

$$F_n = \left[\frac{\varphi^n}{\sqrt{5}} \right] \tag{6}$$

where φ is the golden ratio.

For the same reasons as in `fib4`, this algorithm also runs in constant time and space.

The approximate behaviour of this algorithm is the same as that of `fib4`.

Regarding the algorithmic concepts, `fib5` illustrates the concept of the optimization of a closed-form formula for speed-up in addition to the algorithmic concepts illustrated by `fib4`.

Algorithm 12 `fib6(`n`)`: An algorithm to compute the nth Fibonacci number F_n using the power of a certain 2x2 matrix, computed iteratively by repeated multiplication.

```
def fib6(n:int) -> int:
    n0, n = n, abs(n)
    if n == 0:
        r = 0
    else:
        m = [[1, 0], [0, 1]]
        for _ in range(1, n):
            m = mat_mul_opt(m)
        r = m[0][0]
    if n0 < 0:
        return negafib(n, r)
    return r
```

3.6 Algorithm fib6: The Power of a Certain 2x2 Matrix via Iteration

`fib6` uses the power of a certain 2x2 matrix for the Fibonacci sequence [9, 11]:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1} = \begin{pmatrix} F_n & F_{n-1} \\ F_{n-1} & F_{n-2} \end{pmatrix} \tag{7}$$

where $n \geq 2$. Then, F_n is the largest element of the resulting 2x2 matrix.

The time complexity analysis here is similar to that of `fib3`. With constant-time arithmetic, the time complexity is $O(n)$. Unlike `fib2`, this algorithm keeps only a single 2x2 matrix rather than a table of all the previous values, so its space complexity is $O(1)$, as in `fib3`.

With non-constant time arithmetic, the additions range in time complexity from $A(\lg F_1)$ to $A(\lg F_n)$. The sum of these additions leads to the time complexity of $O(n^2)$ in bit operations. The space complexity is $O(n)$ in the number of bits, since each of the four matrix entries holds $O(n)$ bits.

Regarding the algorithmic concepts, `fib6` illustrates (simple) matrix algebra and iteration over closed-form equations.

Algorithm 13 `fib7(`n`)`: An algorithm to compute the nth Fibonacci number F_n using the power of a certain 2x2 matrix, computed recursively by repeated squaring.

```
def fib7(n:int) -> int:
    n0, n = n, abs(n)
    if n == 0:
        r = 0
    else:
        m = mat_pow_opt_recur(n - 1)
        r = m[0][0]
    if n0 < 0:
        return negafib(n, r)
    return r
```

Algorithm 14 `fib8(`n`)`: An algorithm to compute the nth Fibonacci number F_n using the power of a certain 2x2 matrix, computed iteratively by repeated squaring.

```
def fib8(n:int) -> int:
    n0, n = n, abs(n)
    if n == 0:
        r = 0
    else:
        m = mat_pow_opt_iter(n - 1)
        r = m[0][0]
    if n0 < 0:
        return negafib(n, r)
    return r
```

3.7 Algorithms fib7 and fib8: The Power of a Certain 2x2 Matrix via Repeated Squaring

`fib7` and `fib8` use the same equations used in `fib6` but while `fib6` uses iteration for exponentiation, `fib7` and `fib8` uses repeated squaring for speed-up. Moreover, `fib7` uses a recursive version of repeated squaring while `fib8` uses an iterative version of it.

Repeated squaring reduces time from linear to logarithmic. Hence, with constant time arithmetic, the time complexity is $O(\lg n)$. The space complexity is also logarithmic in n.

With non-constant time arithmetic, the cost is dominated by the multiplications rather than by the additions. Each squaring step multiplies two b-bit numbers, where b grows to $\lg F_n = \Theta(n)$ at the last step. Summing over the $\lg n$ steps gives a geometric series whose last term dominates, so the time complexity is $O(M(n))$ in bit operations, which is $O(n^2)$ under the schoolbook bound $M(b) = O(b^2)$ assumed in Section 2. This agrees with the analyses of `mat_pow_recur` and `mat_pow_iter` given there. The space complexity is linear in the number of bits.

Regarding the algorithmic concepts, `fib7` and `fib8` illustrate (simple) matrix algebra, and repeated squaring over closed-form equations. In addition, `fib7` illustrates recursion while `fib8` illustrates iteration to perform repeated squaring over a matrix.

3.8 Algorithms fib9 and fib10: A Certain Recursive Formula

Both `fib9` and `fib10` use the following formulas for the Fibonacci sequence [11].

$$F_{2n+1} = F_{n+1}^2 + F_n^2 \text{ and } F_{2n} = 2F_{n+1}F_n - F_n^2 \tag{8}$$

where $n \geq 2$, with the base values $F_2 = 1$, $F_1 = 1$, and $F_0 = 0$. Note that memoization is used for speed-up in such a way that only the cells needed for the final result are filled in the memoization table F. In `fib9`, recursion takes care of identifying such cells. In `fib10`, a cell marking phase, using a separate queue of indexes, is used for such identification; then the values for these cells, starting from the base values, are computed in a bottom-up fashion.

These algorithms behave like repeated squaring in terms of time complexity. Hence, with constant time arithmetic, the time complexity is $O(\lg n)$.

Algorithm 15 `fib9(`n`)`: An algorithm to compute the nth Fibonacci number F_n recursively using a certain recursive formula, with memoization.

```
def fib_recur(n:int, F:List[int]) -> int:
    if F[n] is None:
        if n <= 0:
            F[n] = 0
        elif n == 1:
            F[n] = 1
        elif n == 2:
            F[n] = 1
        else:
            k = n >> 1
            f1 = fib_recur(k, F)
            f2 = fib_recur(k + 1, F)
            if is_even(n):
                F[n] = 2 * f1 * f2 - f1 * f1
            else:
                F[n] = f2 * f2 + f1 * f1
    return F[n]

def fib9(n:int) -> int:
    n0, n = n, abs(n)
    r = fib_recur(n, [None] * (n + 1))
    if n0 < 0:
        return negafib(n, r)
    return r
```

Algorithm 16 `fib10`(n): An algorithm to compute the nth Fibonacci number F_n iteratively using the same recursive formula as `fib9`. F is a hash table and B holds the base values $B[0] = 0$, $B[1] = 1$, and $B[2] = 1$.

```
BASE = {0: 0, 1: 1, 2: 1}

def fib10(n:int) -> int:
    n0, n = n, abs(n)

    # find indexes that need F values. None marks an index whose value
    # is not known yet.
    F = {n: None}
    qinx = [n]   # queue of indexes
    while qinx:
        k = qinx.pop() >> 1
        for j in (k, k + 1):
            if j not in F:
                F[j] = None
                qinx.append(j)

    # set base values
    F.update(BASE)

    # fill the indexes that need values. the increasing index order
    # guarantees that F[k >> 1] and F[(k >> 1) + 1] are already known.
    for k in sorted(F.keys()):
        if k in BASE:
            continue
        k2 = k >> 1
        f1, f2 = F[k2], F[k2 + 1]
        if is_even(k):
            F[k] = 2 * f2 * f1 - f1 * f1
        else:
            F[k] = f2 * f2 + f1 * f1

    r = F[n]
    if n0 < 0:
        return negafib(n, r)
    return r
```

Only $O(\lg n)$ of the values $F[k]$ are ever needed, but the two algorithms store them differently: `fib10` keeps exactly those in a hash table, so its space complexity is $O(\lg n)$, whereas `fib9` allocates an array of $n+1$ cells and fills only $O(\lg n)$ of them, so its space complexity is $O(n)$. This is the array vs. hash table distinction noted at the start of Section 3.

With non-constant time arithmetic, the cost is again dominated by the multiplications in the doubling formulas rather than by the additions, since the last step multiplies $\Theta(n)$-bit numbers. As in `fib7` and `fib8`, summing over the $\lg n$ steps gives a time complexity of $O(M(n))$ in bit operations, which is $O(n^2)$ under the schoolbook bound. The values stored take $O(n)$ bits in both algorithms.

Regarding the algorithm concepts, these algorithms illustrate recursion vs. iteration, top-down vs. bottom-up processing and/or dynamic programming, implementation of a recursive relation, and careful use of a queue data structure to eliminate unnecessary work in the bottom-up processing.

3.9 Algorithm fib11: Yet Another Recursive Formula

`fib11` uses the following formulas for the Fibonacci sequence [11].

$$F_{2n+1} = F_{n+1}^2 + F_n^2 \text{ and } F_{2n} = F_{n+1}^2 - F_{n-1}^2 \tag{9}$$

where $n \geq 2$. The case for $n = 2$ needs to be handled as a base case of recursion to prevent an infinite loop. Note that memoization is also used for speed-up.

The time and space complexity analyses are as in `fib10`, which uses the same hash table structure.

Regarding the algorithmic concepts, this algorithm is again similar to `fib9` and `fib10`.

3.10 Algorithm fib12: Yet Another but Simpler Recursive Formula

`fib12` uses the following formula for the Fibonacci sequence

$$F_n = [\varphi F_{n-1}] \tag{10}$$

where $n \geq 3$. It is formula 64 in Vajda [10] and formula 73 in Dunlap [3], both stated in the equivalent reindexed form $F_{n+1} = [\varphi F_n]$ for $n \geq 2$, and it

Algorithm 17 `fib11(`n`)`: An algorithm to compute the nth Fibonacci number F_n iteratively using yet another recursive formula. F and B are as in `fib10`.

```
BASE = {0: 0, 1: 1, 2: 1}

def fib11(n:int) -> int:
    n0, n = n, abs(n)

    # find indexes that need F values. None marks an index whose value
    # is not known yet. negative indexes are never needed: they arise
    # only for k <= 1, which the base values already cover.
    F = {n: None}
    qinx = [n]  # queue of indexes
    while qinx:
        k = qinx.pop() >> 1
        for j in (k - 1, k, k + 1):
            if j >= 0 and j not in F:
                F[j] = None
                qinx.append(j)

    # set base values
    F.update(BASE)

    # fill the indexes that need values. the increasing index order
    # guarantees that F[(k >> 1) - 1], F[k >> 1] and F[(k >> 1) + 1]
    # are already known.
    for k in sorted(F.keys()):
        if k in BASE:
            continue
        k2 = k >> 1
        f1, f2, f3 = F[k2 - 1], F[k2], F[k2 + 1]
        if is_even(k):
            F[k] = f3 * f3 - f1 * f1
        else:
            F[k] = f3 * f3 + f2 * f2

    r = F[n]
    if n0 < 0:
        return negafib(n, r)
    return r
```

Algorithm 18 `fib12(`n`)`: An algorithm to compute the nth Fibonacci number F_n iteratively using a certain recursive formula built on the golden ratio.

```
def fib12(n:int) -> int:
    n0, n = n, abs(n)
    sqrt_5 = math.sqrt(5)
    phi = float(1 + sqrt_5) / 2
    if n == 0:
        r = 0
    elif n == 1:
        r = 1
    elif n == 2:
        r = 1
    else:
        r = 1
        for _ in range(3, n + 1):
            r = round(phi * r)
        r = int(r)
    if n0 < 0:
        return negafib(n, r)
    return r
```

is listed with those attributions in Knott's collection of Fibonacci and golden ratio formulae [8].

The identity follows from Eq. 3 in one step. Substituting $F_n = (\varphi^n - \psi^n)/\sqrt{5}$ and using $\psi - \varphi = -\sqrt{5}$,

$$\varphi F_{n-1} - F_n = \frac{\varphi^n - \varphi\psi^{n-1} - \varphi^n + \psi^n}{\sqrt{5}} = \frac{\psi^{n-1}(\psi - \varphi)}{\sqrt{5}} = -\psi^{n-1}. \qquad (11)$$

So φF_{n-1} misses F_n by exactly ψ^{n-1}. Since $|\psi| \approx 0.618$, we get $|\psi^{n-1}| < \frac{1}{2}$ precisely when $n \geq 3$, which is where the rounding starts to be correct: at $n = 2$, $\varphi F_1 \approx 1.618$ rounds to 2 rather than to $F_2 = 1$. Note that this bound is on the formula itself, in exact arithmetic; the value of n beyond which `fib12` returns approximate results is set instead by the floating-point precision it uses.

Note that although the golden ratio is only the limit value of the ratio of the consecutive Fibonacci numbers, Eq. 11 shows that the formula is not merely asymptotic: the rounding is correct for every $n \geq 3$, not just for large n. Our implementation returns exact results from $n = 3$ to $n = 78$ and approximate ones beyond that, like the algorithms `fib4` and `fib5`, due to the use of floating-point arithmetic.

The time and space complexity analyses are as in `fib3`. It is easy to implement a version of this algorithm where both recursion and memoization are used.

Regarding the algorithmic concepts, this algorithm illustrates the iterative version of a recursive formula.

4 Results

We now present the results of a small-scale experimental analysis done on a high-end laptop computer (model: MacBook Pro, operating system: macOS Sierra 10.12.6, CPU: 2.5 GHz Intel Core i7, main memory: 16GB 1600 MHz DDR3).

For each algorithm, we measure its runtime in seconds (using the `perf_counter()` method from the `time` module in the Python standard library). We ran each algorithm 10,000 times and collected the runtimes. The reported runtimes are the averages over these repetitions. We also computed the standard deviation over these runtimes. We use the standard deviation to report "the coefficient of variability" (CV) (the ratio of the standard deviation to the average runtime of an algorithm over 10,000 repetitions), which gives an idea on the variability of the runtimes.

We report the results in four groups, moving the focus towards the fastest algorithms as n gets larger:

1. The case $0 \leq n \leq 30$ is small enough to run all algorithms, including the slowest algorithm `fib1`.

2. The case $0 \leq n \leq 70$ excludes the slowest algorithm `fib1`. In this case all algorithms are exact. On our experimental setting, `fib4` and `fib5` start returning approximate results after $n = 70$, and `fib12` after $n = 78$.

3. The case $0 \leq n \leq 900$ excludes the approximation algorithms, i.e., `fib4`, `fib5`, and `fib12`. The upper bound 900 is also roughly the upper bound beyond which the recursive algorithms start exceeding their maximum recursion depth limit and error out.

4. The case $0 \leq n \leq 10,000$ focuses on the fastest algorithms only, excluding the slow algorithms, the recursive algorithms, and the approximate algorithms.

Table 1: Algorithms

Algorithm	Runtime in constant-time ops	Runtime in bit ops
`fib1`	$O(\varphi^n)$	$O(\varphi^n)$
`fib2`	$O(n)$	$O(n^2)$
`fib3`	$O(n)$	$O(n^2)$
`fib4`	$O(1)$	$O(1)$
`fib5`	$O(1)$	$O(1)$
`fib6`	$O(n)$	$O(n^2)$
`fib7`	$O(\lg n)$	$O(M(n)) = O(n^2)$
`fib8`	$O(\lg n)$	$O(M(n)) = O(n^2)$
`fib9`	$O(\lg n)$	$O(M(n)) = O(n^2)$
`fib10`	$O(\lg n)$	$O(M(n)) = O(n^2)$
`fib11`	$O(\lg n)$	$O(M(n)) = O(n^2)$
`fib12`	$O(n)$	$O(n^2)$

The bit-operation column uses the schoolbook bounds $A(b) = O(b)$ and $M(b) = O(b^2)$ from Section 2. A subquadratic multiplication improves every $O(M(n))$ entry: the Python implementations use Karatsuba multiplication, for which $M(b) = O(b^{1.585})$. The $O(1)$ entries for `fib4` and `fib5` count the floating-point arithmetic only; any algorithm that returns F_n as an exact integer must write $\Theta(n)$ bits, so $\Omega(n)$ is a lower bound on the output alone.

Each plot in the sequel has two axes: The x-axis is n, as in the index of F_n; and the y-axis is the average runtime in seconds over 10,000 repetitions.

To rank the algorithms in runtime, we use the sum of all the runtimes across all the n range. These rankings should be taken as directionally correct as the variability in runtimes, especially for small n, makes it difficult to assert a definite ranking.

4.1 Results with all algorithms

The results with $0 \leq n \leq 30$ in Fig. 1 mainly show that the exponential-time algorithm `fib1` significantly dominates all others in runtime. This algorithm is not practical at all to use for larger n to compute the Fibonacci numbers. The ratio of the slowest runtime to the fastest runtime, that of `fib5`, is about four orders of magnitude, 14,000 to be exact on our experimental setting.

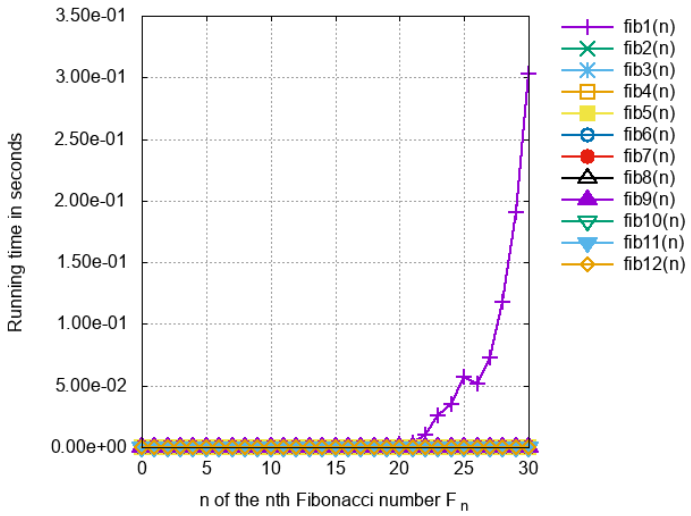

Figure 1: Results until `fib1` takes too much time.

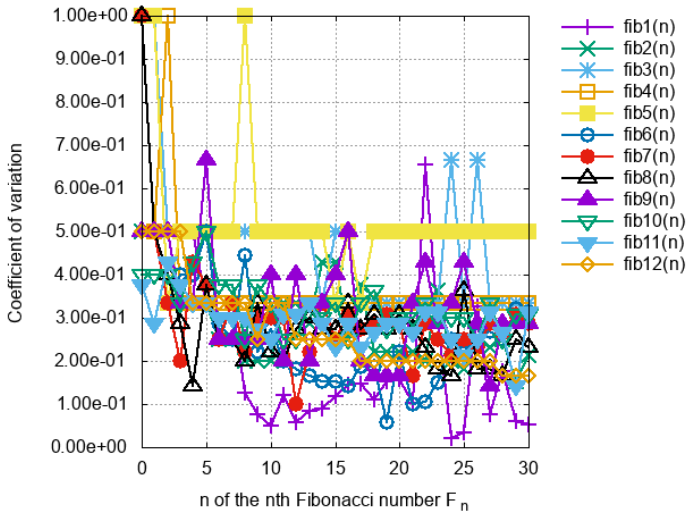

Figure 2: Coefficient of variation for above.

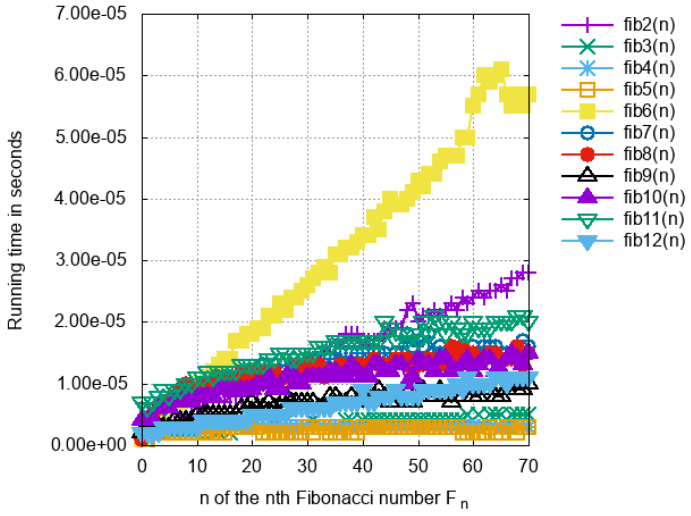

Figure 3: Results with all algorithms (excl. `fib1`) returning exact results.

The CV results in Fig. 2 are not very reliable at this scale, since the runtimes are so small. They still fall below 35% for the most part, with an outlier at 50% for `fib5`. For the slowest algorithm `fib1`, the CV is largely around 5%, which is expected given its relatively large runtime.

4.2 Results with all fast algorithms returning exact results

The results with $0 \leq n \leq 70$ in Fig. 3 exclude the slowest algorithm `fib1`. The results show that `fib6` is now the slowest among the rest of the algorithms; its runtime grows linearly with n, while the runtimes of the rest grow sublinearly. The algorithms implementing the closed-form formulas run in almost constant time.

The algorithms fall into the following bands of comparable runtime, given in increasing order; within a band the differences are within the measurement noise:

- `fib5`, `fib4`, `fib3`;
- `fib12`, `fib9`;

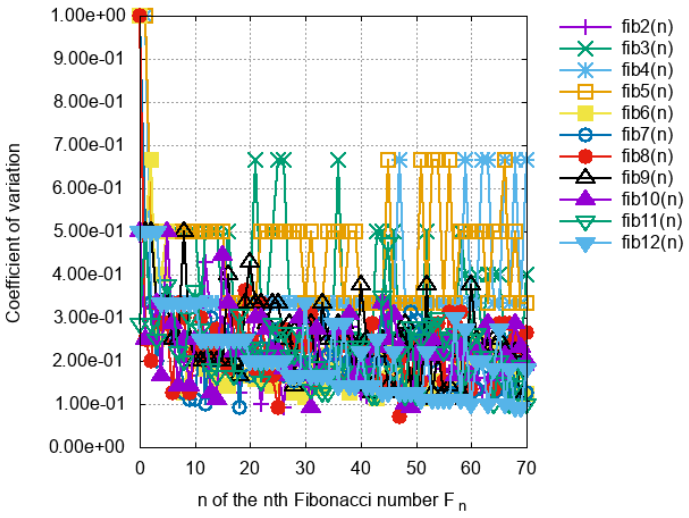

Figure 4: Coefficient of variation for above.

- `fib10`, `fib8`, `fib7`, `fib2`, `fib11`;
- `fib6`.

These runtimes show that the algorithms implementing the closed-form formulas are the fastest. The ratio of the slowest runtime, that of `fib6`, to the fastest runtime, that of `fib5`, is about an order of magnitude, 13 to be exact on our experimental setting.

The CV results in Fig. 4 are similar to those of the first case above.

4.3 Results within safe recursion depth

The results with $0 \leq n \leq 900$ in Fig. 5 exclude the slowest algorithm `fib1`. The upper bound of n is roughly the limit beyond which recursive algorithms fail due to the violation of their maximum recursion depth.

The results show that `fib6` followed by `fib2` are the slowest among the rest of the algorithms; their runtimes grow linearly with n, though the slope for `fib2` is much smaller. The remaining algorithms grow sublinearly. Again, the algorithms implementing the closed-form formulas run in almost constant time.

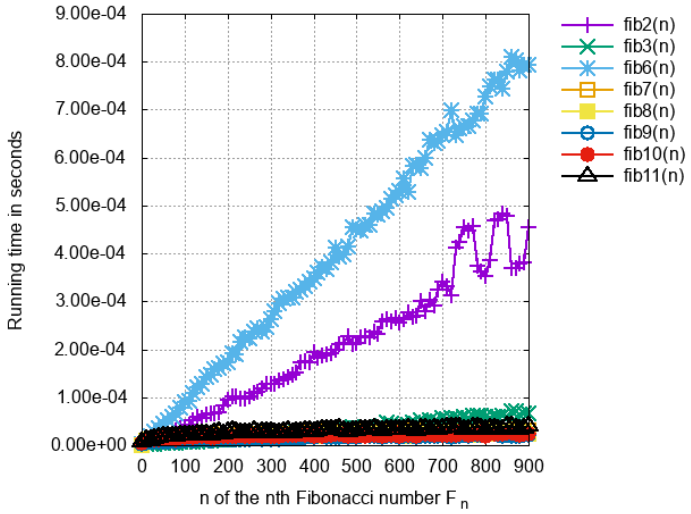

Figure 5: Results until recursive algorithms hit too deep a recursion depth.

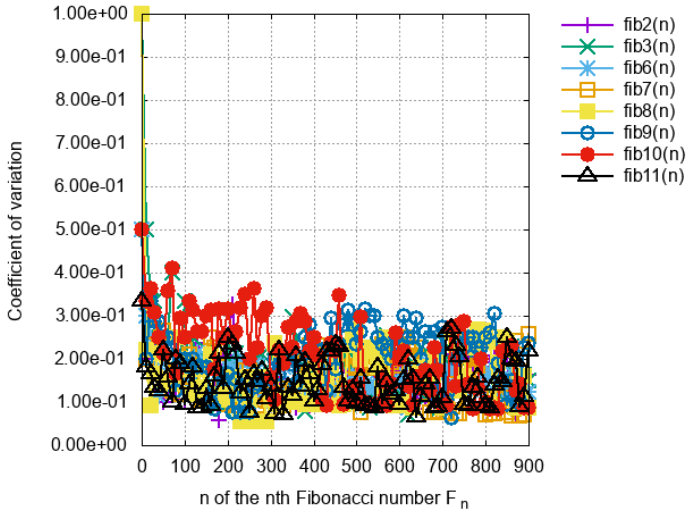

Figure 6: Coefficient of variation for above.

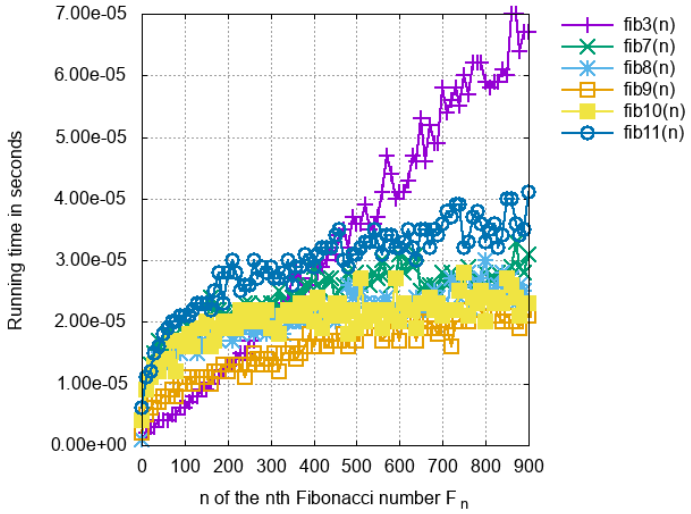

Figure 7: Results until recursive algorithms hit too deep a recursion depth. Faster algorithms focus.

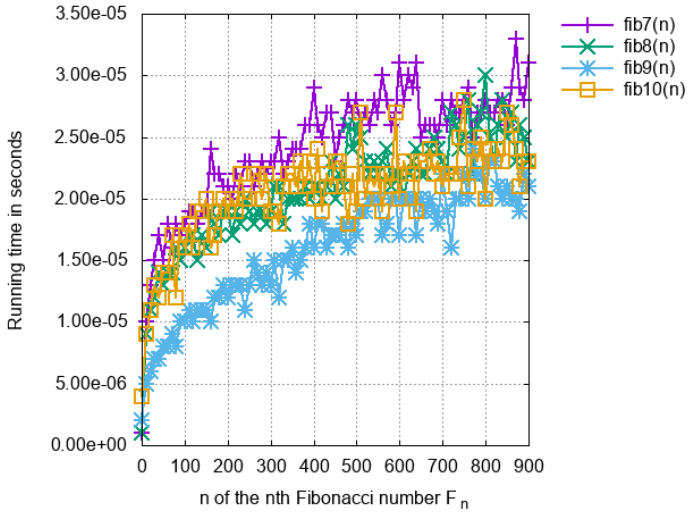

Figure 8: Results until recursive algorithms hit too deep a recursion depth. Even faster algorithms focus.

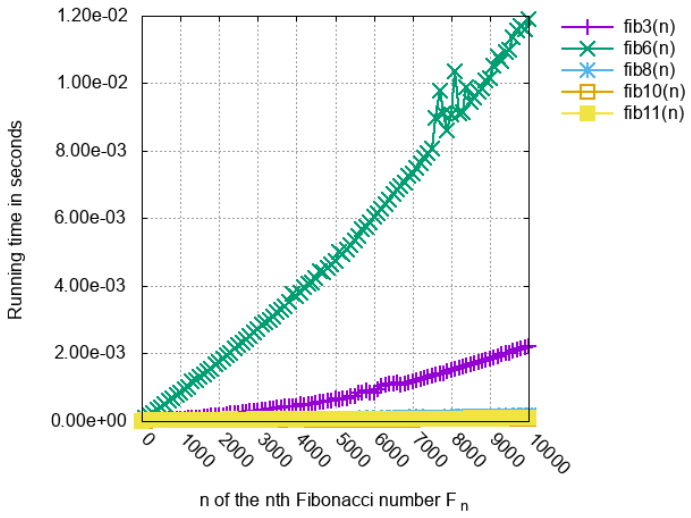

Figure 9: Results using only iterative algorithms with exact results.

The algorithms group as follows in terms of their runtimes in increasing runtime order:

- `fib9`, `fib10`, `fib8`, `fib7`, `fib11`, `fib3`;
- `fib2`;
- `fib6`.

The ratio of the slowest runtime, that of `fib6`, to the fastest runtime, that of `fib9`, is about two orders of magnitude on our experimental setting.

Fig. 7 and Fig. 8 zoom in on the faster algorithms, excluding the slowest algorithms from Fig. 5.

The CV results in Fig. 6 are all below 20%, which is reasonably small given that the runtimes are now larger.

4.4 Results with the fastest iterative and exact algorithms

The results with $0 \leq n \leq 10{,}000$ in Fig. 9 exclude all slow algorithms, all recursive algorithms, and all inexact algorithms, those that do not return exact results.

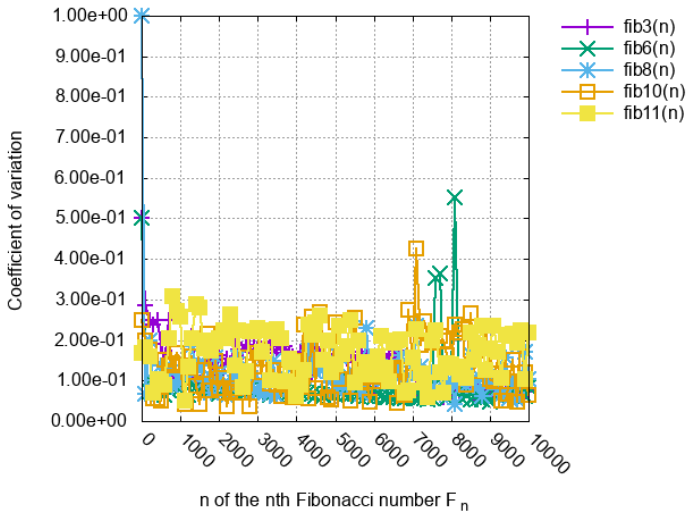

Figure 10: Coefficient of variation for above.

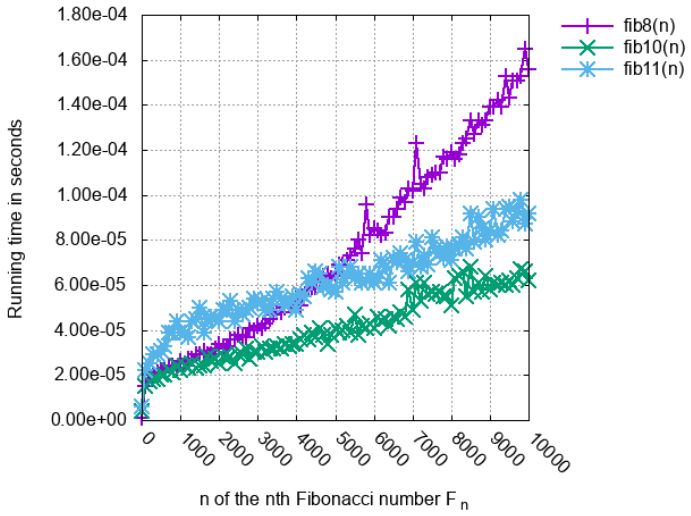

Figure 11: Results using only iterative algorithms with exact results. Faster algorithms focus.

In this range where n gets very large, the fastest and slowest algorithms are `fib10` and `fib6`, respectively. The algorithms group as follows in terms of their runtimes in increasing runtime order:

- `fib10`, `fib11`, `fib8`;
- `fib3`;
- `fib6`.

The ratio of the slowest runtime, that of `fib6`, to the fastest runtime, that of `fib10`, is about two orders of magnitude, 130 to be exact on our experimental setting.

Fig. 11 zooms in on the fastest algorithms, excluding the slowest algorithms from Fig. 9.

The CV results in Fig. 10 have all converged to values below 20%, which is again reasonably small given how much larger the runtimes are at this range of n.

5 Conclusions

The Fibonacci numbers are well known and simple to understand. We have selected from the technical literature [1, 6, 9, 11] twelve methods of computing these numbers. Some of these methods are recursive formulas and some others are closed-form formulas. We have translated each method into an algorithm and implemented it in the Python programming language. Each algorithm takes in n and returns the nth Fibonacci number F_n.

Though simple, these algorithms illustrate a surprisingly large number of concepts from the algorithms field, from top-down vs. bottom-up dynamic programming through to recursion depth, and probably more. Section 1 lists all of them, each with the algorithms that illustrate it. The simplicity of these algorithms then becomes a valuable asset in teaching introductory algorithms, in that students can focus on the concepts rather than on the complexity of the algorithms themselves.

We have also presented a small-scale experimental analysis of these algorithms to further enhance their understanding. The analysis reveals a couple of interesting observations, e.g., how two algorithms that implement seemingly similar recursive formulas may have widely different runtimes, how space usage can affect time, how or why recursive algorithms cannot have

too many recursive calls, when approximate algorithms stop returning exact results, etc. The results section explains these observations in detail with plots.

Probably the simplest fast algorithm to implement is `fib3`, the one that uses constant space. However, the fastest algorithm, especially for large n, turns out to be `fib10`, the one that implements a recursive algorithm with a logarithmic number of iterations. When $n \leq 70$ where all algorithms return exact results, the fastest algorithms are `fib4` and `fib5`, the ones that implement the closed-form formulas.

The slowest algorithm is of course `fib1`, the one that implements probably the most well-known recursive formula, which is usually also the definition of the Fibonacci numbers. Memoization does speed it up immensely but there is no need to add complexity when simpler and faster algorithms also exist.

We hope that this paper can serve as a useful resource for students learning and teachers teaching the basics of algorithms. All the programs used for this study are at [2].

6 Homework Questions

We now list a number of homework questions for the students. These questions should help improve the students' understanding of the algorithmic concepts further.

1. Try to simplify the algorithms further, if possible.
2. Try to optimize the algorithms further, if possible.
3. Replace O with Θ in the complexity analyses, if possible.
4. Prove the time and space complexities for each algorithm.
5. Improve, if possible, the time and space complexities for each algorithm. A trivial way is to use the improved bounds on $M(\cdot)$.
6. Reimplement the algorithms in other programming languages. The simplicity of these algorithms should help in the speed of implementing in another programming language.

7. Derive an empirical expression for the runtime of each algorithm. This can help derive the constants hidden in the O-notation (specific to a particular compute setup).

8. Rank the algorithms in terms of runtime using statistically more robust methods than what this paper uses.

9. Find statistically more robust methods to measure the variability in the runtimes and/or to bound the runtimes.

10. Explain the reasons for observing different real runtimes for the algorithms that have the same asymptotic time complexity.

11. Learn more about the concepts related to recursion such as call stack, recursion depth, and tail recursion. What happens if there is no bound on the recursion depth?

12. Explain in detail how each algorithm illustrates the algorithmic concepts that Section 1 assigns to it, and argue for or against any assignment you disagree with.

13. Design and implement a recursive version of the algorithm `fib11`.

14. Design and implement versions of the algorithms `fib4` and `fib5` that use rational arithmetic rather than floating-point arithmetic. For the non-rational real numbers, use their best rational approximation to approximate them using rational numbers at different denominator magnitudes.

15. Find other formulas for the Fibonacci numbers and implement them as algorithms.

16. Prove the formula used by the algorithm `fib12` by induction on n, without appealing to the closed-form formula that Section 3 uses for it.

17. Use the formula involving binomial coefficients, $F_n = \sum_k \binom{n-k-1}{k}$, to compute the Fibonacci numbers. This will also help teach the many ways of computing binomial coefficients.

18. Design and implement multi-threaded or parallelized versions of each algorithm. Find out which algorithm is the easiest to parallelize and which algorithm runs faster when parallelized.

19. Measure, on your own machine, the largest n for which `fib4`, `fib5` and `fib12` still return exact results. Why is the limit of `fib12` different from that of `fib4` and `fib5`? Of the limits reported in Section 3, which would change under a different math library, and which follows from the IEEE 754 double precision format alone and so cannot?

20. Recompute the bit-operation column of Table 1, which assumes the schoolbook bound $M(b) = O(b^2)$, for Karatsuba multiplication, $M(b) = O(b^{1.585})$, and for an FFT-based multiplication. Find which algorithms change places, and whether the resulting ranking matches the measured runtimes for large n.

21. Explain why `fib9` and `fib10` differ in space complexity though they compute the same values from the same formula, then modify `fib9` so that it too uses $O(\lg n)$ space.

22. Explain what is missed by testing every algorithm in this paper against another algorithm from it, namely the one used by `fib3`, and design a test that catches it.

23. Repeat the measurements for negative n, which every algorithm accepts but the experiments never cover, and find whether the rankings change and whether they should.

24. Analyse the algorithms that the repository [2] has added beyond these twelve, place them in Table 1, and find which of the concepts of Section 1 each illustrates.

25. Responsibly update Wikipedia using the learnings from this study and/or your extensions of this study.

References

[1] T. Cormen, C. S. R. Rivest, and C. Leiserson. *Introduction to Algorithms.* McGraw-Hill Higher Education, 2nd edition, 2001.

[2] A. Dasdan. Fibonacci Number Algorithms. https://github.com/alidasdan/fibonacci-number-algorithms.

[3] R. A. Dunlap. *The Golden Ratio and Fibonacci Numbers*. World Scientific, 1997. Formula 73 in Appendix A3.

[4] M.Forišek. Towards a Better Way to Teach Dynamic Programming. Olympiads in Informatics. vol. 9. 2015.

[5] D.Goldberg. *What Every Computer Scientist Should Know About Floating-Point Arithmetic*. ACM Computing Surveys, Vol 23(1), 5–48, 1991.

[6] R. L. Graham, D. E. Knuth, and O. Patashnik. *Concrete Mathematics: A Foundation for Computer Science*. Addison-Wesley, 2nd edition, 1994.

[7] J. L. Holloway. *Algorithms for Computing Fibonacci Numbers Quickly*. MSc Thesis. Oregon State Univ. 1988.

[8] R. Knott. Fibonacci and Golden Ratio Formulae. https://r-knott.surrey.ac.uk/fibonacci/fibFormulae.html.

[9] J. Sondow and E. Weisstein. *Fibonacci number*. In From MathWorld–A Wolfram Web Resource. Wolfram. Accessed January 2018. http://mathworld.wolfram.com/FibonacciNumber.html.

[10] S. Vajda. *Fibonacci and Lucas Numbers, and the Golden Section: Theory and Applications*. Ellis Horwood, 1989; reprinted by Dover, 2008. Formula 64.

[11] Wikipedia contributors. Fibonacci sequence. *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Fibonacci_sequence. Accessed January 2018, when the article was titled "Fibonacci number".

[12] Wikipedia contributors. Computational complexity of mathematical operations. *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Computational_complexity_of_mathematical_operations. Accessed March 2018.

[13] Wikipedia contributors. Dynamic programming. *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Dynamic_programming. Accessed May 2018.

[14] Wikipedia contributors. Recursion (computer science). *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Recursion_(computer_science). Accessed May 2018.

[15] Wikipedia contributors. Time complexity. *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Time_complexity. Accessed May 2018.

[16] Wikipedia contributors. Call stack. *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Call_stack. Accessed May 2018.

[17] Wikipedia contributors. Tail call. *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Tail_call. Accessed May 2018.

[18] Wikipedia contributors. Floating-point arithmetic. *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Floating-point_arithmetic. Accessed May 2018.

[19] Wikipedia contributors. Closed-form expression. *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Closed-form_expression. Accessed May 2018.